

Wipro Network Management System

COGNINENT – AI POWERED NETWORK FAULT AND ROOT CAUSE ANALYSIS SYSTEM

Project Team

230701138 — Kamalesh SP

230701158 — Kumaran R

230701414 — Ganesh M

Guided By

[MENTOR NAME]

Challenges in Monitoring and Managing Network Infrastructure

Modern network environments contain multiple devices, interfaces, and continuously changing operational information. As the network grows, manually observing device health, interface conditions, and network events becomes increasingly difficult.

Limited Device Visibility

Network administrators need a centralized view of managed network devices and their operational status. When information is distributed across individual devices, obtaining an overall view of network health becomes difficult.

Manual Interface Monitoring

Network devices contain multiple interfaces whose operational states can change. Checking individual interfaces manually increases monitoring effort and makes continuous observation difficult.

Telemetry Management

Network operational information — interface status, device information, and performance metrics — needs to be collected, processed, and stored systematically.

Delayed Fault Detection

An interface or device can transition from a normal state to a failed state. Without continuous monitoring and processing, such conditions may not be identified immediately.

Unstructured Alarm Tracking

Identifying a fault is only the first step. The fault needs to be recorded, tracked while active, and cleared when the underlying condition is resolved.

Inconsistent Fault Tracking

Active, recurring and resolved network conditions can be difficult to track systematically and the effort required for continuous monitoring.

 **CORE PROBLEM:** How can network devices, interfaces, telemetry, and faults be monitored through a single centralized Network Management System?
Flow: Multiple Network Devices → Distributed Information → Manual Monitoring → Delayed Fault Detection → Operational Difficulty

WIPRO NMS — Centralized Network Monitoring, Telemetry and Alarm Management

WIPRO NMS is a centralized Network Management System designed to bring network device information, interface monitoring, telemetry processing, and alarm management into one unified platform. The system addresses each of the five core challenges through dedicated capabilities.



Device Management

Maintains centralized information about managed network devices: device identity, hostname, IP address, device type, operating information, and device association with interfaces. Provides a centralized inventory and operational view of managed network devices.



Interface Management

Maintains information about individual network interfaces: interface identity, name, IP address, associated device, and operational status. Provides visibility into the operational state of individual interfaces.



Telemetry Monitoring

Processes network monitoring information including interface operational status, device status, CPU metrics, memory metrics, interface traffic, and other network telemetry. Converts network activity into structured monitoring information.



Alarm Management

Manages abnormal network conditions through a structured alarm lifecycle: alarm creation, updates, active alarm deduplication, alarm reopening where applicable, alarm clearing, and first-seen/last-seen tracking. Provides structured visibility into network faults and their current state.



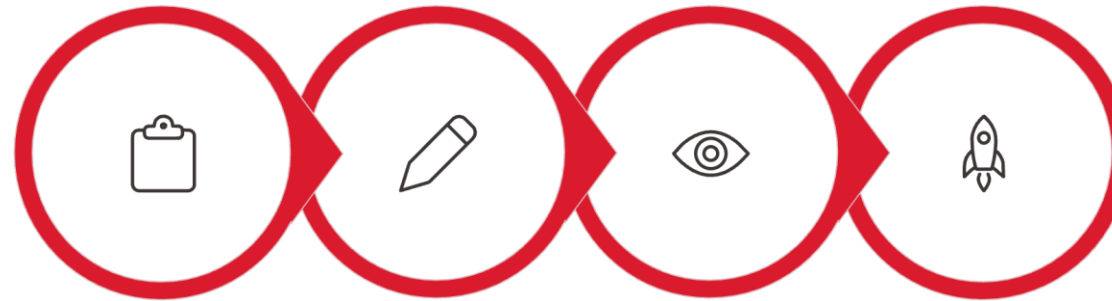
Centralized Dashboard

The web interface provides a unified operational view of devices, interfaces, telemetry, alarms, and network status.



AI-Powered Network Intelligence

Analyzes network data and alarms to detect anomalies, perform root-cause analysis, reduce alarm noise, predict potential failures, and explain incidents to the operator.



Plan

Create

Review

Deploy

WIPRO NMS — System Components and Working

The six core components of WIPRO NMS work together to deliver end-to-end network monitoring, from simulation and telemetry ingestion to alarm handling, persistence, and dashboard visualization.

1. Network Simulation — GNS3 + FRRouting

GNS3 provides the virtual lab environment used to build and validate the topology before deployment. The simulated routers (R1, R2, R3) run FRRouting, advertise routes, exchange traffic, and expose real interface states such as link up/down, administrative status, and routing adjacency changes. This lets the NMS observe realistic device behavior without requiring physical hardware.

Role: Create the network topology, emulate routers and links, generate operational events such as interface failures or route changes, and provide a controlled environment for testing monitoring and alarm logic.

Example data source: R1 ↔ R2 ↔ R3, with each router exposing interface counters, reachability, and protocol state that the NMS can poll or receive via telemetry.

2. Monitoring & Data Collection — SNMP / Telemetry

Network devices expose operational data that the NMS collects continuously. SNMP polling retrieves structured values such as interface ifOperStatus and ifAdminStatus, traffic counters (inOctets/outOctets), error counters (CRC, drops, discards), CPU utilization, memory usage, and device uptime. Telemetry updates can also be pushed at intervals to capture near-real-time changes in operational state.

Flow: Network Device → SNMP/Telemetry Collector → Parsed Monitoring Data → NMS backend

What is collected: interface status, bandwidth utilization, packet errors, CPU/memory load, system uptime, and interface traffic trends used to detect degradation or outages.

3. Backend & REST API — Java 21 + Spring Boot 4.0.8

The backend acts as the central application layer. Spring Boot services expose REST endpoints for device inventory, interface status, telemetry history, and alarm records. Service components validate incoming requests, map raw telemetry into domain models, apply business rules, and coordinate persistence through repository classes. The API layer also powers dashboard queries such as device summaries, active alarms, and time-series telemetry views.

Flow: Client / Frontend → REST API → Controller Layer → Service Layer → Repository Layer → PostgreSQL

Specific responsibilities: normalize device and interface identifiers, look up related records, enforce alarm deduplication rules, compute current operational state, and expose filtered data for list views, detail pages, and chart panels.

4. Telemetry Processing

Telemetry received by the system is processed by backend services in a multi-step pipeline. The backend first parses the payload, identifies the metric name and timestamp, associates it with the correct device and interface, and then applies metric-specific interpretation rules. For example, interface_oper_status = 1 is translated to UP, while interface_oper_status = 0 is translated to DOWN. Similar logic can be applied to CPU thresholds, memory pressure, and traffic spikes to determine whether the observation is normal or requires attention.

Processing steps: receive telemetry → validate payload → identify metric → map to device/interface → persist raw and processed values → evaluate state transition → emit monitoring result.

Example:

interface_oper_status = 1 → Interface UP → normal state recorded

interface_oper_status = 0 → Interface DOWN → fault condition detected and forwarded to alarm logic

5. Alarm Engine

The alarm engine manages detected network faults through a structured lifecycle: NORMAL → FAULT DETECTED → ALARM OPENED → ALARM UPDATED WHILE ACTIVE → CONDITION RECOVERED → ALARM CLEARED. When a monitored condition crosses a defined rule or threshold, the engine creates an alarm with a deterministic identifier so repeated polls do not create duplicates. If the fault persists, the existing alarm is updated with the latest timestamps, severity, and state. When the device or interface recovers, the alarm is closed and marked cleared.

Capabilities: create alarms from fault events, suppress duplicate active alarms, update existing alarms with fresh evidence, reopen alarms if a cleared condition recurs, and track first-seen, last-seen, and cleared timestamps for auditability.

Alarm Identifier: IF-DOWN-{deviceId}-{interfaceId}

Trigger example: Interface DOWN → alarm opened; interface remains DOWN on the next poll → existing alarm updated; interface returns UP → alarm and state reset to normal.

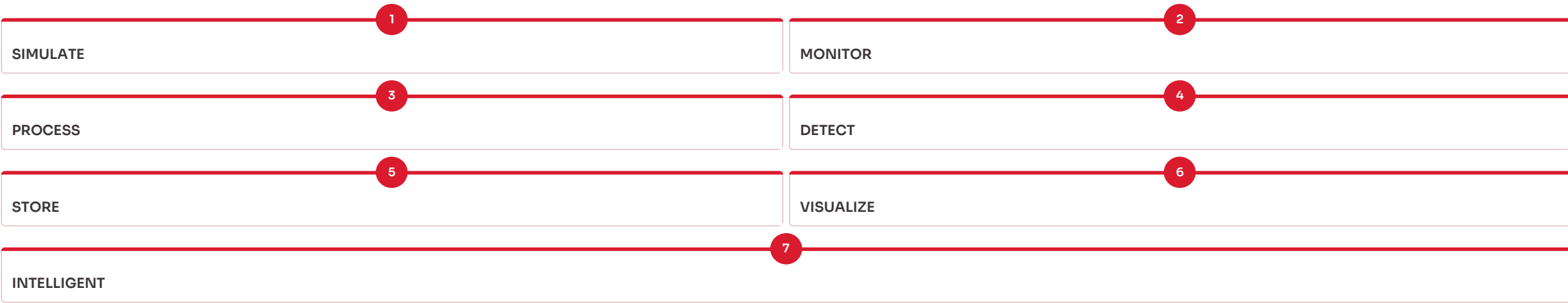
6. AI-Powered Network Intelligence

Analyzes network data and alarms to detect anomalies, perform root-cause analysis, reduce alarm noise, predict potential failures, and explain incidents to the operator.

How it works: The AI layer consumes telemetry, alarm history, topology context, and operational state from the existing backend and database. It correlates device, interface, and alarm patterns to identify abnormal behavior, suppress duplicate or low-value alerts, and rank likely failure causes. It can also learn from historical incidents to surface early warning signals before a fault becomes user-visible.

Integration points: reads parsed telemetry from the processing pipeline, enriches alarms with probable causes, uses the device/interface model for dependency analysis, and writes findings back through the API for dashboard display.

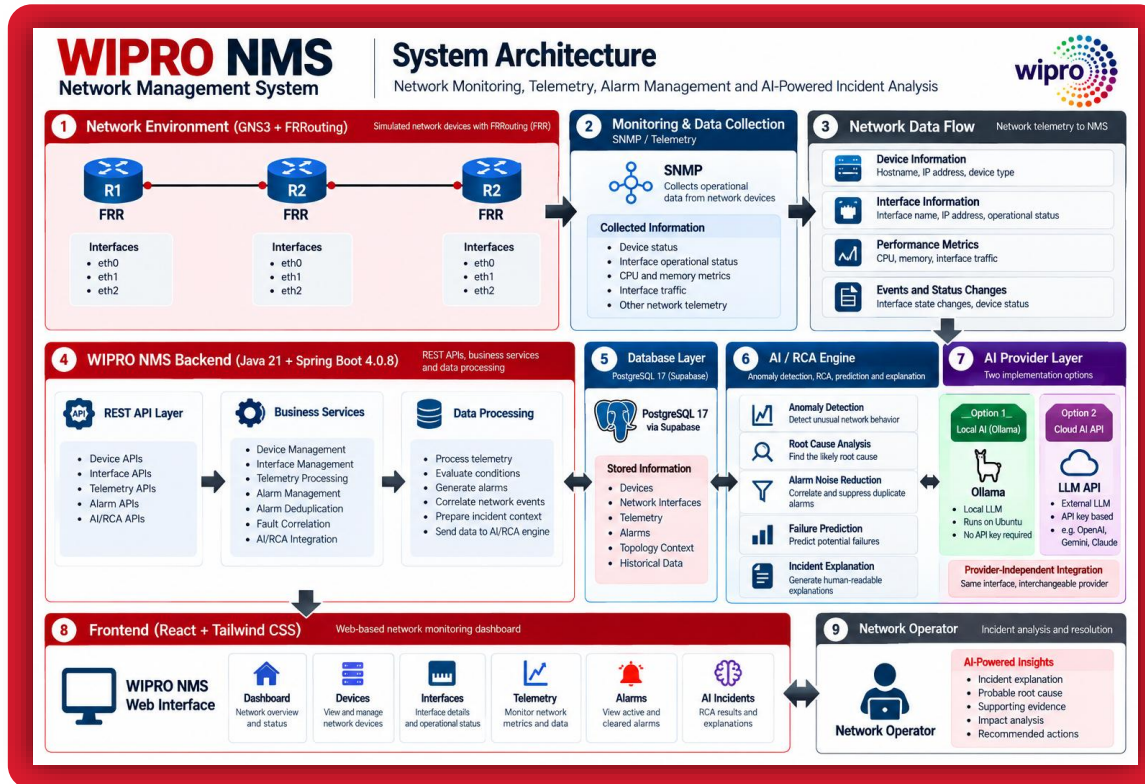
Outputs: anomaly scores, probable root cause, incident summaries, suggested remediation steps, and predictions of degraded interfaces or devices.



WIPRO NMS — System Architecture

SYSTEM ARCHITECTURE

GNS3 / FRRouting → SNMP / Telemetry → Spring Boot Backend → PostgreSQL Database → REST API → React Dashboard



Technology Summary

Network

GNS3 ▪ FRRouting ▪ SNMP
▪ AI Provider Layer ▪ API

Backend

Java 21 ▪ Spring Boot 4.0.8 ▪
Spring Data JPA ▪ Hibernate

Database

PostgreSQL 17 ▪ Supabase ▪
HikariCP

Frontend

React ▪ Tailwind CSS

Existing System and Its Limitations

In a conventional or distributed monitoring environment, administrators may need to inspect individual network devices and interfaces to understand their operational state. Information can be distributed across different devices or monitoring sources, making centralized visibility difficult.

Existing Approach

Network Devices → Individual Monitoring → Manual Status Checking → Manual Fault Identification → Separate Information

Limited Centralized Visibility

Network information is not necessarily available through one unified operational interface.

Manual Monitoring Effort

Administrators may need to inspect individual devices and interfaces.

Delayed Fault Awareness

Operational changes may not be identified immediately without continuous monitoring.

Difficult Alarm Tracking

Without structured alarm lifecycle management, active and resolved faults are harder to track.

Limited Data Organization

Telemetry and operational information need to be consistently organized for monitoring and historical review.

Higher Monitoring Effort

As the number of devices and interfaces increases, checking each component individually becomes more difficult.

WIPRO NMS — Improved Approach

Multiple devices → Centralized monitoring → Telemetry processing → Structured alarms → Unified dashboard

Centralized Visibility

All devices and interfaces visible from one unified dashboard.

Automated Monitoring

Continuous telemetry collection replaces manual status checking.

Immediate Fault Detection

Alarm engine detects and records faults as they occur.

Structured Alarm Lifecycle

Alarms are created, updated, and cleared with full timestamp tracking.



WIPRO NMS brings network devices, operational data, telemetry, and alarm management into a centralized monitoring platform.

Methodology and Technology Stack

Development Methodology

01

Requirement Analysis

Identify requirements for device management, interface monitoring, telemetry, alarm management, and dashboard visualization.

02

Network Environment Setup

Build the virtual network topology using GNS3 + FRRouting.

03

Backend Development

Develop the backend using Java 21 + Spring Boot 4.0.8. Implement REST APIs, services, and persistence.

04

Telemetry Processing

Process monitoring information and associate it with the relevant device/interface.

05

Alarm Implementation

Implement: Detection → Creation → Update → Clearing.

06

Database Integration

Persist Devices, Interfaces, Telemetry, and Alarms using PostgreSQL.

07

Frontend Development

Build the monitoring dashboard using React + Tailwind CSS.

08

AI-Powered Network Intelligence

Analyzes network and alarms to detect anomalies, root-cause analysis, reduce alarm noise and explain incidents to the operator.

Technology Stack

Network & Simulation

GNS3 • FRRouting (FRR) • SNMP

Backend

Java 21 • Spring Boot 4.0.8 • Spring Data JPA • Hibernate ORM • Maven

Database

PostgreSQL 17 • Supabase • HikariCP

Frontend

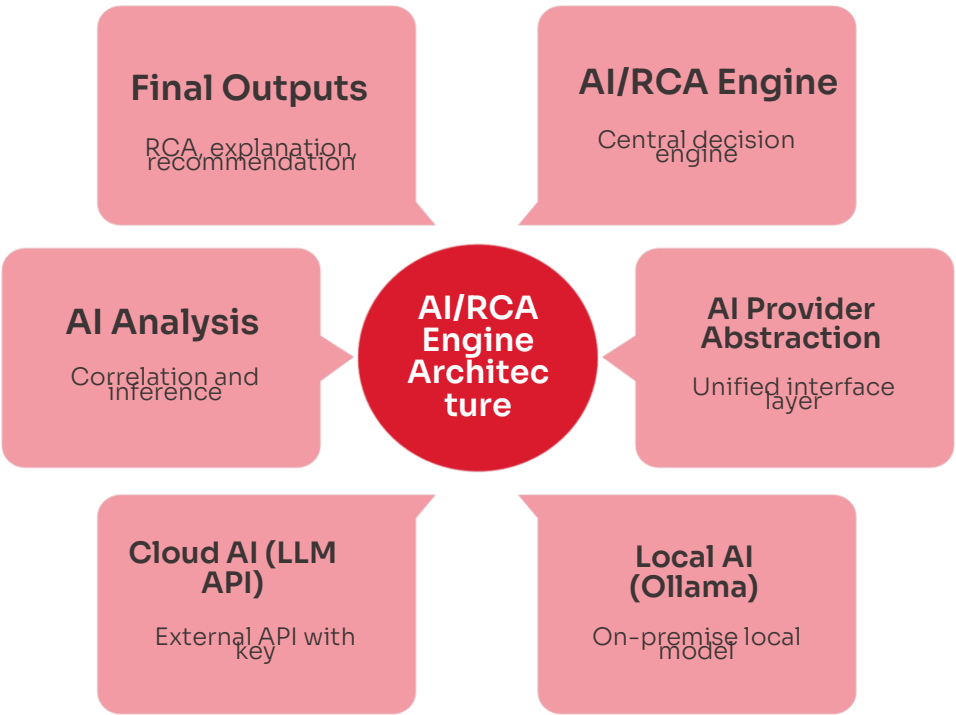
React • Tailwind CSS • REST / JSON

Runtime

Apache Tomcat 11 • Linux / Ubuntu

AI Local/ Provider Layer

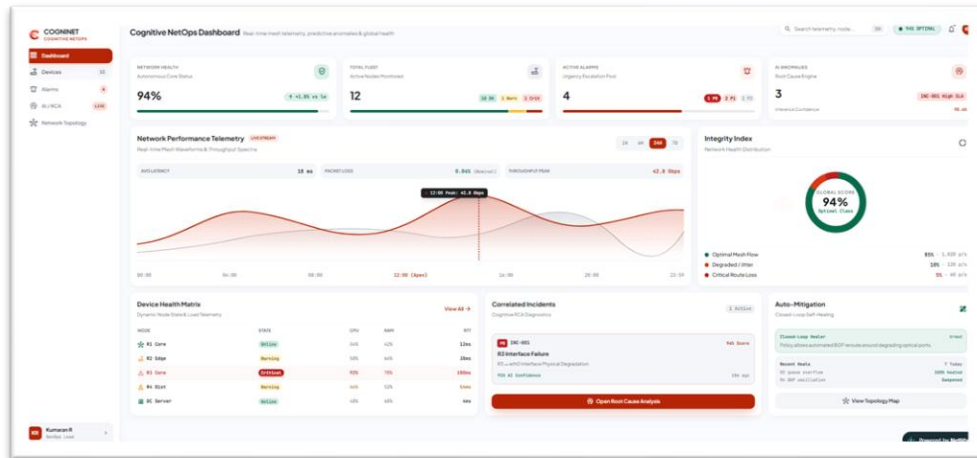
Ollama + Local LLM • External LLM API



WIPRO NMS — Implementation I

Web-Based Network Monitoring Interface

The React frontend consumes REST APIs exposed by the Spring Boot backend and presents processed network information through a unified monitoring dashboard. The interface provides centralized views of network status, device and interface management, and telemetry and alarm monitoring.



Dashboard / Network Overview

Centralized view of network status and monitoring information.

The screenshot shows the 'Managed Devices' interface with a sidebar menu on the left containing 'Dashboard', 'Events', 'Alerts', 'Network Topology', and 'Network Telemetry'. The main content area displays a 'Devices Fleet' section with a 'New Contingent' button and a table of network devices. The table has columns for 'Name', 'Model', 'IP Address', 'Status', 'CPU', 'Memory', 'Latency', 'Loss', 'Interfaces', and 'Actions'.

Name	Model	IP Address	Status	CPU	Memory	Latency	Loss	Interfaces	Actions
Router-Core-01	Core Router	10.240.0.10	CRITICAL	95%	78%	120 ms	0.05%	2 Ports	View
Router-Core-02	Core Router	10.240.0.2	HEALTHY	45%	45%	120 ms	0.05%	2 Ports	View
Router-Edge-01	Edge Router	10.240.0.14	HEALTHY	55%	45%	120 ms	0.05%	2 Ports	View
Router-Spine-01	Distribution Switch	10.240.0.10	HEALTHY	45%	45%	120 ms	0.05%	2 Ports	View
DC-Server-Cluster-Alpha	DC Server Cluster	10.240.0.10	HEALTHY	45%	45%	120 ms	0.05%	2 Ports	View
Router-Phy-01	Phy Router	10.240.0.10	HEALTHY	25%	25%	120 ms	0.05%	2 Ports	View

Device and Interface Management

View and manage registered network devices and interfaces.

React

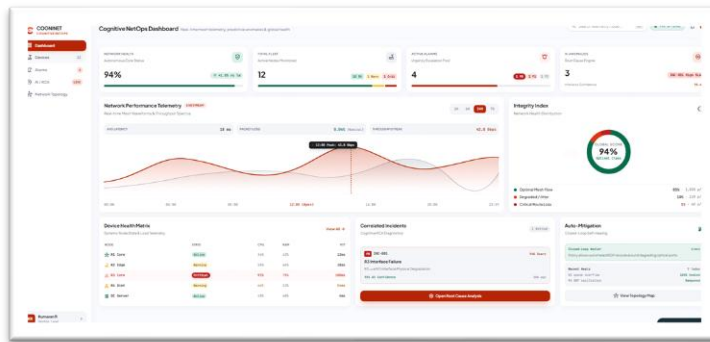
Tailwind CSS

REST API

WIPRO NMS —Implementation II

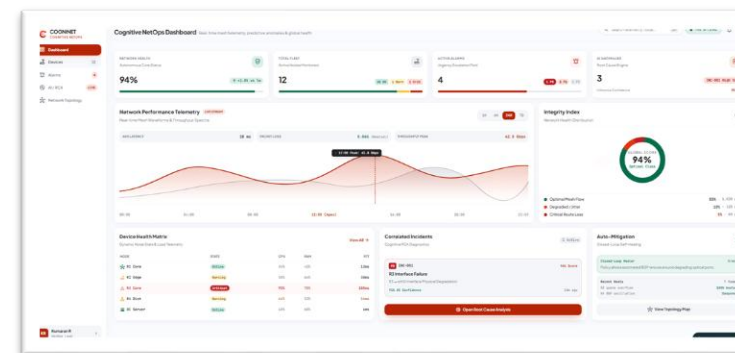
REST APIs, Telemetry Processing and Alarm Management

The Spring Boot backend acts as the central application layer, providing REST APIs and application services for device management, interface management, telemetry processing, and alarm management. The backend receives requests and monitoring information, applies business logic, and communicates with persistent storage.



Alarm Management / API Response

Backend alarm creation, status management, and clearing.



Telemetry Processing

Backend processing and persistence of network monitoring information.

Java 21

REST API

Hibernate

Spring Boot 4.0.8

JPA

PostgreSQL 17

THANK YOU

WIPRO NMS CAPSTONE PROJECT

Network Management System — Capstone Project

Project Team

230701138 — Kamalesh SP
230701158 — Kumaran R
230701414 — Ganesh M

Guided By

[MENTOR NAME]

Technology Summary

GNS3 ▪ FRRouting ▪ SNMP ▪ Java 21 ▪
Spring Boot 4.0.8 ▪ Spring Data JPA ▪
Hibernate ORM ▪ Maven ▪ Apache Tomcat
11 ▪ PostgreSQL 17 ▪ Supabase ▪ HikariCP ▪
React ▪ Tailwind CSS ▪ REST / JSON ▪
LOCAL AI LLM ▪ ExternalAPI

[RAJALAKSHMI ENGINEERING COLLEGE] · [COMPUTER SCIENCE AND ENGINEERING]